- ***Bug rates*** —If you see lots of bugs being generated as you're delivering good code, that's an indication your code isn't maintainable. You'll usually see bug rates parallel poor code coverage, high issues from static analysis, and high MTTR and lead times.
- ***Recidivism*** —Recidivism directly impacts MTTR and lead time because as things go backward in the workflow, you're effectively doing the same things more than once. If a developer says something is done but it ends up getting rejected by QA, that development work needs to be done again, impacting your delivery times. High recidivism typically parallels high bug rates and can be affected by all the other metrics we've discussed relating to maintainability.
- ***Velocity*** —All of the metrics discussed in this chapter potentially affect velocity. It's possible to have a steady cadence of delivery on a terrible codebase. In that case you'll have a team that's not reaching their potential output. If you see a consistent velocity with terrible maintainability metrics, then you should think about how you can make your codebase more maintainable to get more done.

If your application is very maintainable, you should be able to deliver to your consumers consistently and with high quality. Delivery is the first half of the equation; now you have to make sure that what you're delivering is what your consumers want and something they can use.

8.4. Measuring usability

There are several different "ilities" that demonstrate how usable your application is or how good an experience you're giving to your consumers. The big ones we'll focus on are outlined in figure 8.12.

Figure 8.12. Once something has been delivered, you need to measure it to make sure it's good.